

Digital Asset System

Backend Design & Logic Mapping

Area-wide GIS assets (drainage, sewer, water, roads, electrical) on top of the Property Survey platform.
Generated: 2026-07-08

1. Purpose

Extend the property-survey backend into a full digital asset register ("digital twin") of an area. Beyond parcels/buildings, the system now models point, line and polygon infrastructure across four categories, with an admin shapefile-ingest workflow, a layered map, and a field survey/correction loop for surveyors.

2. Core data model

A generic, admin-configurable GIS schema. Adding a new asset type is data (a new AssetLayer row), not code.

- Project — the top-level container an admin creates; shapefile uploads, asset features and property surveys are scoped to it via project_id. The admin uploads all data into a chosen project.
- AssetCategory — grouping for the legend/report (e.g. Drainage & Sewer).
- AssetLayer — a layer/asset-type definition: geometry_type (POINT/LINESTRING/POLYGON), map style, and an attribute_schema that drives survey forms and shapefile mapping.
- AssetFeature — one geometry instance. geom is geometry(Geometry,4326) (true WGS84); cached length_m / area_sqm are geodesic. Lifecycle: STAGED !' VERIFIED !' PUBLISHED, plus FLAGGED / REJECTED.
- AssetUpload — one import batch (shapefile-zip or GeoJSON); the unit the admin reviews and publishes.
- AssetSurvey — a surveyor's field observation or proposed correction (new geometry / attributes), reviewed and applied by an admin.
- AssetPhoto — Cloudinary-hosted field photos on a feature/visit.

3. Seeded asset catalog

Category	Layer	Geometry
Drainage & Sewer	Sewer Line	LINESTRING
Drainage & Sewer	Storm Drain	LINESTRING
Drainage & Sewer	Manhole	POINT
Water Supply	Water Pipeline	LINESTRING
Water Supply	Valve	POINT
Water Supply	Fire Hydrant	POINT
Water Supply	Overhead Tank	POLYGON
Roads & Transport	Road Centerline	LINESTRING
Roads & Transport	Streetlight	POINT
Roads & Transport	Culvert	POINT
Electrical	Electric Pole	POINT
Electrical	Transformer	POINT
Electrical	LT Line	LINESTRING
Electrical	HT Line	LINESTRING

4. Admin workflow — upload !' verify !' publish

- Admin selects a project + layer and uploads a zipped

shapefile (.shp/.dbf/.prj) or a GeoJSON (project_id is required).

- Server parses with shpjs and reprojects from the file's CRS (from .prj) to WGS84.
- Only geometries matching the layer's geometry_type are staged; mismatches are counted and skipped.
- Features land as STAGED under an AssetUpload; the admin reviews them on the map (GET uploads/:id/features).
- Verify !' features VERIFIED. Publish !' features PUBLISHED (live to surveyors). Reject/Delete discards the batch.

5. Surveyor workflow — survey & correct

- Surveyor opens the map; nearby published assets load via GET features/nearby (GPS + radius).
- For each asset they VERIFY, CORRECT_GEOMETRY (redraw), CORRECT_ATTRIBUTE, or FLAG a flaw, with condition + notes + photos.
- The proposal is stored on AssetSurvey and the feature is marked FLAGGED — it is NOT auto-applied.
- Admin approves !' corrected geometry/attributes copied onto the feature, status back to PUBLISHED. Reject leaves it unchanged.

6. Layered map

GET /api/assets/map returns every active layer with its features grouped as GeoJSON plus the layer's draw style and attribute schema — the frontend renders one toggleable Leaflet overlay per layer. Filters: ward_id, bbox (viewport), status (PUBLISHED default; ALL for admin review).

7. Geometry & CRS notes

- New asset geometry is

stored natively in EPSG:4326 and returned via ST_AsGeoJSON — no transform on read.

- Legacy note: existing Polygons store UTM 44N (EPSG:32644) coordinates and are transformed 32644!4326 on read; the two datasets stay separate.
- A GiST index on AssetFeatures.geom backs bbox and nearby (ST_DWithin) queries.

8. API map

Method	Path	Purpose
POST	/api/projects	Create a project (admin)
GET	/api/projects	List projects (with asset counts)
GET	/api/projects/:id/summary	Project asset inventory summary
GET	/api/assets/categories	Catalog: categories + their layers
GET/POST	/api/assets/layers	List / create asset layers (admin)
PUT/DELETE	/api/assets/layers/:id	Edit / archive a layer (admin)
POST	/api/assets/layers/:id/uploads	Upload shapefile-zip or GeoJSON !' stage (admin)
GET	/api/assets/uploads	List import batches
GET	/api/assets/uploads/:id/features	Staged features as GeoJSON (map review)
POST	/api/assets/uploads/:id/verify	Mark batch verified (admin)
POST	/api/assets/uploads/:id/publish	Publish batch !' live for surveyors
POST	/api/assets/uploads/:id/reject	Reject batch
GET	/api/assets/map	Layered map: all layers + features grouped
GET	/api/assets/layers/:id/features	One layer's features (GeoJSON, bbox/ward filters)
POST	/api/assets/features	Manually add a feature (draw on map)
PUT/DELETE	/api/assets/features/:id	Edit geometry/attrs / soft-delete
GET	/api/assets/features/nearby	Assets near a surveyor's GPS
POST	/api/assets/features/:id/survey	Surveyor observation / correction
POST	/api/assets/surveys/:id/approve	Apply a correction to the feature (admin)
POST	/api/assets/features/:id/photos	Attach field photos (Cloudinary)
GET	/api/assets/stats	Per-layer inventory (counts/length/area/status)
GET	/api/assets/reports/ward/:wardId	Ward asset report PDF
GET	/api/assets/reports/project/:projectId	Project asset report PDF
GET	/api/assets/docs/design.pdf	This design document

9. Gaps & next steps

- Frontend: add a layer-toggle control + shapefile upload UI (website GIS dashboard) and asset survey screens (mobile app). Backend endpoints are ready.
- Auto-assign ward_id / polygon_id to imported features via a spatial join (ST_Contains) instead of a single ward

per batch.

- Large imports: switch the per-feature INSERT loop to a batched/COPY insert for very large shapefiles.
- Attribute mapping UI: let the admin map .dbf columns to the layer's attribute_schema during upload.
- Topology validation (snap manholes to sewer ends, connectivity checks) as a verification aid.
- Versioning/audit trail on feature geometry edits (currently corrections are applied in place).